# TW1 — Foundational DevOps Skills

## DevOps Lab Submission

**Name:** Aparna Nair

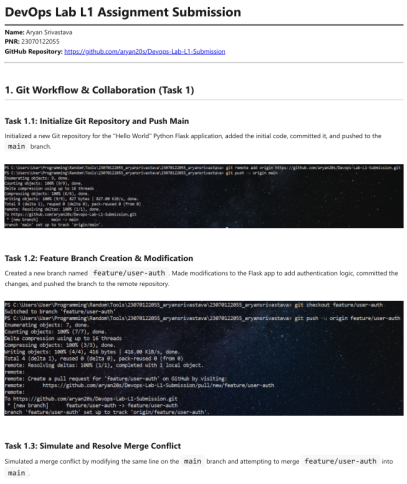
**PRN:** 23070122040

# 1. Git Workflow & Collaboration

Set up the Flask Hello World project in Git, create the feature branch, and resolve the merge conflict.

## Initialize repository and push main

```
git init
git branch -M main
git add app.py
git commit -m "Initial Flask Hello World app"
git remote add origin https://github.com/<your-username>/<your-repo>.git
git push -u origin main
```



DevOps Lab L1 Assignment Submission

**Name:** Aryan Srivastava
**PNR:** 23070122055
**GitHub Repository:** https://github.com/aryan20lx/Devops-Lab-L1-Submission

1. Git Workflow & Collaboration (Task 1)

Task 1.1: Initialize Git Repository and Push Main

Initialized a new Git repository for the "Hello World" Python Flask application, added the initial code, committed it, and pushed to the `main` branch.

Task 1.2: Feature Branch Creation & Modification

Created a new branch named `feature/user-auth`. Made modifications to the Flask app to add authentication logic, committed the changes, and pushed the branch to the remote repository.

Task 1.3: Simulate and Resolve Merge Conflict

Simulated a merge conflict by modifying the same line on the `main` branch and attempting to merge `feature/user-auth` into `main`.

Git repository initialized and the main branch pushed to the remote repository.

## Create feature branch

```
git checkout -b feature/user-auth
# Make a small change in app.py
git add app.py
git commit -m "Add feature branch message"
git push -u origin feature/user-auth
```

## Resolve merge conflict

```
git checkout main
# Edit the same line changed on the feature branch
git add app.py
git commit -m "Update Hello World message on main"
git merge feature/user-auth
# Remove conflict markers and keep the final code
git add app.py
git commit -m "Resolve merge conflict"
git push origin main
```

**Merge Conflict Detected in Code:**

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
<<<<<<< HEAD
    print("Hello world accessed!")
=======
    print("User authenticated successfully!")
>>>>>>> feature/user-auth
    return 'Hello World!'

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

**Manual Conflict Resolution and Push:** Manually resolved the conflict, committed the resolution, and successfully pushed the updated `main` branch.



**2. Jira Project & Issue Tracking (Task 2)**

**Task 2.1: Scrum Project Creation**

Created a new Scrum project in Jira Cloud named "Hello World".

Merge conflict in app.py and the resolved changes pushed to main.

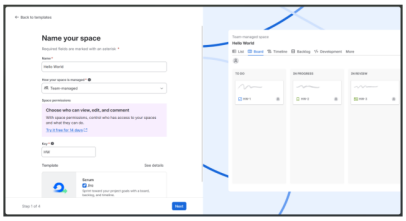
## 2. Jira Project & Issue Tracking

Create a Scrum project in Jira Cloud named Hello World.

### Create issues

```
Story: Implement User Authentication Feature
Task: Setup Flask Environment
Bug: Login Page Displays Error
```

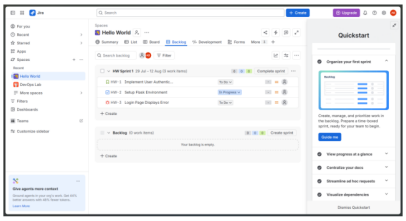
Move Setup Flask Environment from To Do to In Progress on the project board.




**Task 2.2: Issue Creation**

Created three distinct issues as requested:

- **Story:** Implement User Authentication Feature
- **Task:** Setup Flask Environment
- **Bug:** Login Page Displays Error




**Task 2.3: Sprint Board Management**

Jira project board with the required issues and task status.

## 3. Docker & Jenkins Freestyle Project

### Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

## Build and run

```
docker build -t hello-world-flask:latest .
docker run -d --name hello-flask -p 5000:5000 hello-world-flask:latest
curl http://localhost:5000
docker ps
```

**Docker Build and Run:**



**Verification (cURL localhost:5000):** The application successfully returns "Hello World!" confirming the container is running correctly.



**Task 3.2: Jenkins Freestyle Project Setup**

Set up Jenkins using a Docker container and configured a Freestyle project to pull the Git repository and execute a shell script build step.

Docker image built and container started; Flask responds on localhost:5000.

## Jenkins Freestyle project

Configure Jenkins to check out the Git repository and run a build step that lists the workspace files.

```
ls -la
# On Windows, use:
dir
```

**Successful Build Output:** Triggered a build. The console output confirms successful cloning and workspace listing.



Jenkins Freestyle build console output showing the repository checkout and workspace listing.